

LLM-Enhanced Graph-Based Intrusion Detection

A Parallel GNN + LLM Framework with Bidirectional Refinement

Supervised by Fouad Al Tfaily | CESI LINEACT / Lebanese University

04/05/2026

Research Goal

Current IDS approaches are half-blind. GNNs read network structure but carry no semantic knowledge of what attack patterns mean. LLMs know cybersecurity concepts but cannot process graph topology natively. The goal is to build a system that sees both dimensions simultaneously: a GNN encodes the structural fingerprint of network traffic, an LLM encodes its semantic meaning enriched by a knowledge graph built from the same data, and both representations are fused for final attack classification. The novelty confirmed by literature survey is that no prior work applies bidirectional iterative GNN-LLM interaction with joint classification to network intrusion detection.

Step 1: Graph Construction from the Dataset

From any NetFlow-based IDS dataset (NF-ToN-IoT, CIC-IDS, UNSW-NB15, etc.), construct a directed communication graph as follows. Each unique IP address becomes a node. Each aggregated flow between two IPs becomes a directed edge carrying attributes: flow count, total bytes, average duration, most common protocol, most common destination port, and attack label.

Concretely: group the CSV by (source IP, destination IP, attack label). Each unique combination produces one edge. This collapses millions of flow records into thousands of meaningful graph edges while preserving the structural communication patterns that characterize each attack type.

The resulting graph is what the GNN operates on. A DDoS attack produces a hub-and-spoke structure with many sources converging on one victim. A scanning attack produces a star structure with one source reaching many destinations. These structural fingerprints are exactly what the GNN learns to recognize.

Step 2: Knowledge Graph Construction

The Knowledge Graph (KG) is built from the same CSV, programmatically, with no external knowledge base required at this stage. The process has three sub-steps.

2.1 Aggregate into Triples

Group the dataset by (source IP, destination IP, attack label). For each unique group compute: flow count, average bytes, average duration, most common protocol, most common destination port. Each group becomes one KG triple:

source_IP $\xrightarrow{\text{relation_name}}$ destination_IP | frequency, bytes, duration, protocol, port

2.2 Discretize Numerical Attributes

Raw numbers are not meaningful to an LLM. Convert all numerical attributes into semantic labels. For each attribute (flow count, average bytes, average duration), collect all values within that attack type only, sort them, and split into thirds: bottom third becomes **low**, middle third becomes **medium**, top third becomes **high**. This is done per attack type separately because each attack operates at a different natural scale. Protocol and port are already categorical and require no conversion.

2.3 Map Attack Labels to Relation Names

Replace the attack label with a descriptive verb that describes what the attacker did. This makes each triple readable as a natural language sentence, which is essential for the LLM to reason over it.

Attack Label	Relation Name
Scanning	initiated_scan
DDoS	launched_ddos
DoS	launched_dos
Password	attempted_brute_force
Injection	performed_injection
XSS	executed_xss
Backdoor	established_backdoor
Benign	communicated_with

The final KG is one single connected graph. The same IP node can appear with multiple edge types if it participated in multiple traffic types. An example triple in natural language: *“192.168.1.1 initiated_scan on 10.0.0.2, frequency: high, bytes: low, duration: low, protocol: TCP, port: 22.”*

Step 3: Parallel Dual-Encoder

The same underlying network traffic is now represented in two forms: a communication graph for the GNN, and a knowledge graph of semantic triples for the LLM. Both encoders run in parallel on the same data.

GNN path: A Graph Attention Network (GAT) processes the communication graph. IP nodes aggregate information from their neighbors through attention-weighted message passing. The output is a structural embedding $\mathbf{h}_v \in \mathbb{R}^d$ per node, capturing topology, centrality, and community behavior.

LLM path: KG triples are serialized into natural language sentences and passed to a pre-trained language model (BERT-class or instruction-tuned). The LLM produces a semantic embedding $\mathbf{s}_v \in \mathbb{R}^d$ per flow, capturing the meaning of the traffic enriched by cybersecurity context.

Fusion: Both embeddings are concatenated and passed to a lightweight attention-based classifier that produces the final attack label. The classifier learns per-sample attention weights that determine how much to rely on structural vs. semantic evidence. DDoS detection will be structurally dominant. Injection and XSS detection will be semantically dominant. These weights are interpretable by design.

Step 4: Bidirectional Feedback Loop (Phase 2)

Phase 1 runs both encoders independently and fuses at the end. Phase 2 introduces an iterative feedback loop between them.

After each GAT message-passing round, identify flows where the GNN is uncertain: flows with high entropy in their predicted probability distribution. These are the hard cases, typically novel attack types or ambiguous traffic patterns. Serialize those specific flows into natural language and route them to the LLM. The LLM produces a semantic judgment score for each flagged flow. This score is injected back into the GAT as an attention bias on the relevant edges, re-weighting which neighbors the GNN attends to in the next message-passing round. The loop repeats until convergence.

This is the core novelty. It transforms the system from a static parallel architecture into one where structural and semantic reasoning mutually refine each other. The GNN tells the LLM where to look. The LLM tells the GNN what it found. No prior work applies this to intrusion detection.

Implementation note: build and validate Phase 1 first. Once the parallel dual-encoder produces clean results and baselines are established, add the feedback loop as an extension. This is the correct order.

Step 5: Validation

Compare against four baselines:

1. GNN only (structural, no LLM)
2. LLM only (semantic, no GNN)
3. Sequential (GNN then LLM, or LLM then GNN)
4. Parallel Phase 1 (dual-encoder, no feedback loop)
5. Full system with feedback loop (Phase 2)

Metrics: accuracy, F1 per attack type, with particular attention to novel or low-sample attack types where the LLM’s pretrained knowledge should provide the largest gain over GNN-only.

Novelty Statement

The literature survey confirmed zero prior art combining all three of: (1) bidirectional iterative GNN-LLM interaction at both training and inference, (2) attention-fusion joint classification of structural and semantic embeddings, and (3) application to network intrusion detection with an internally constructed knowledge graph from flow data. Each element exists separately in the literature. The combination in this domain does not.